

Bash-Based Tool for Server Intrusion Detection and System Hardening

Introduction

In response to the growing need for automated, repeatable security assessments on Linux servers, I developed a comprehensive Bash script that consolidates multiple critical security checks into a single, efficient workflow. The script is designed to be run regularly, either manually, or via a cron job to provide a clear snapshot of their server's security posture. It prioritizes logical sequencing: first checking for pending system updates/upgrades, then enumerating network services, performing rootkit and malware scans, and finally analyzing authentication logs. Every command is prefixed with `sudo` to ensure it has the necessary privileges, and the script gracefully handles missing tools by displaying clear installation instructions rather than failing abruptly.

System Update Verification

The script begins by refreshing the package lists using `'apt update'` and `'apt upgrade'`. This step is foundational because many attacks exploit known vulnerabilities that have already been patched in newer software versions. By highlighting upgradable packages, the script immediately alerts the administrator to apply security updates before proceeding with deeper checks. This proactive approach ensures that the server is not being evaluated while running outdated, vulnerable code, which could produce misleadingly positive results from the other detection tools.

Network and Service Enumeration

Understanding what services are exposed to the network is essential for reducing the attack surface. The script uses `'ss -tulpn'` to display all listening TCP and UDP ports along with the associated process names and PIDs. This reveals any unexpected or unauthorized services that an attacker might have installed, such as reverse shells, backdoors, or rogue web servers. Additionally, if Docker is installed on the system, the script runs `'docker ps'` to list running containers. Containerized environments can become points of compromise if they are configured with excessive privileges or contain vulnerable images; enumerating them helps maintain visibility across both host and container workloads.

Rootkit and Malware Detection

To uncover hidden persistent threats, the script employs two specialized tools: rkhunter (Rootkit Hunter) and ClamAV. Rkhunter scans the system for known rootkits, backdoors, and other malicious artifacts by comparing system binaries, kernel modules, and configuration files against trusted signatures. It runs with the `--skip-keypress` flag to allow fully automated execution without user interaction. Following the rootkit scan, the script initiates a full-filesystem malware scan using ClamAV. Before scanning, it updates virus definitions with `freshclam` (if available) to ensure the latest signatures are used. The scan is performed recursively on the root directory (`/`), with sensible exclusions for virtual filesystems (`/proc`, `/sys`, `/dev`, `/run`) to avoid errors and reduce scan time. Any infected files are logged to `/tmp/clamav_scan.log` for later review. Together, rkhunter and ClamAV provide a layered defense: signature-based rootkit detection complemented by broad-spectrum malware scanning.

Authentication and Log Analysis

The final section of the script focuses on auditing user activity and system events. It first displays the last 20 successful logins using `'last -a'`, followed by the last 20 failed login attempts using `'lastb'`. These lists can reveal patterns of brute-force attacks, unauthorized access, or suspicious login times. The script then extracts the last 50 system warnings from the journal using `'journalctl --priority=warning --since "24h ago"'`, providing a concise summary of recent kernel or service errors that might indicate compromise attempts. Next, it searches `/var/log/auth.log` for the string "Failed password" to show the most recent authentication failures, giving immediate insight into active attacks. It also extracts recent sudo commands from the same log, allowing the administrator to see which elevated-privilege actions have been executed recently. Finally, the script queries fail2ban in detail: it retrieves the list of all active jails and then iterates through each one, displaying the jail's status, filter, actions, and currently banned IP addresses. This granular fail2ban output is particularly valuable for assessing whether automated defenses are effectively blocking repeated intrusion attempts.

Conclusion

This Bash script delivers a thorough, automated security assessment that can be run on any Debian-based Linux server. By combining update checks, network enumeration,

rootkit detection, malware scanning, and comprehensive log analysis into a single tool, it significantly reduces the time and expertise required to perform regular security audits. The script is designed for production environments, as it is non-destructive, handles missing dependencies gracefully, and produces clear, actionable output.